
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Direction Générale des Études Technologiques

Institut Supérieur des Études Technologiques de Sidi Bouzid

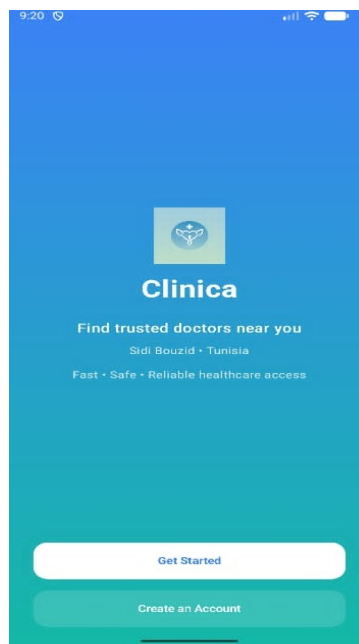
RAPPORT DE PROJET DE FIN DE SEMESTRE

Parcours : Ingénierie des Applications Mobiles

Clinica

Application Mobile Android

Optimisation de la Recherche des Professionnels de Santé



Kotlin • Firebase • OpenStreetMap • Figma

Réalisée par :

Nada Nasri

Encadrant : M. Wael Bouazizi

Année universitaire 2025 – 2026

Table des matières

Table des matières	2
Table des figures	3
Liste des tableaux	4
Introduction Générale	5
Chapitre 1 : Étude Préalable et Présentation du Projet	6
1.1 Présentation du Projet	6
1.2 Problématique	6
1.3 Objectifs du Projet	6
1.4 Étude de l'Existant	7
1.4.1 Description des solutions existantes	7
1.4.2 Tableau comparatif	7
1.5 Méthode de Développement Adoptée	7
Chapitre 2 : Analyse et Spécification des Besoins	9
2.1 Identification des Acteurs	9
2.2 Diagramme de Cas d'Utilisation Global	9
2.3 Besoins Fonctionnels	9
2.3.1 Authentification sécurisée	10
2.3.2 Recherche et Géolocalisation	10
2.3.3 Prise de rendez-vous	10
2.3.4 Messagerie patient-médecin	10
2.4 Besoins Non Fonctionnels	10
Chapitre 3 : Conception du Système	12
3.1 Architecture du Système	12
3.1.1 Vue d'ensemble	12
3.1.2 Architecture en quatre couches	12
3.2 Modélisation des Données Firebase Firestore	12
3.2.1 Collection users	12
3.2.2 Collection doctors	13
3.2.3 Collection appointments	13
3.3 Diagrammes de Conception	13
3.3.1 Diagramme de Classes	13
3.3.2 Diagrammes de Séquence	13
3.4 Prototype Figma	20
Chapitre 4 : Réalisation et Interfaces de l'Application	21
4.1 Environnement de Développement	21
4.1.1 Environnement matériel	21
4.1.2 Environnement logiciel	21
4.2 Technologies Utilisées	21
4.2.1 Kotlin — Langage natif Android	22
4.2.2 Firebase — Backend as a Service	22
4.2.3 OpenStreetMap — Cartographie libre	22

Table des figures

Figure 2.1 · Diagramme de Cas d'Utilisation · Médecin-Patient.....	12
Figure 2.2 · Diagramme de Cas d'Utilisation · Rôle Administrateur.....	13
Figure 3.1 · Architecture Générale du Système Clinica.....	14
Figure 3.2 · Diagramme de Classes · Clinica.....	15
Figure 3.3 · Diagramme de Séquence · Authentification (Login).....	16
Figure 3.4 · Diagramme de Séquence · Discussion Patient-Médecin.....	17
Figure 3.5 · Diagramme de Séquence · Planifier un Rendez-vous.....	18
Figure 3.6 · Diagramme de Séquence · Ajout d'un Médecin (Admin).....	19
Figure 4.1 · Écran d'accueil (Splash Screen) et création de compte.....	26
Figure 4.2 · Écrans des spécialités et liste des médecins.....	27
Figure 4.3 · Carte OpenStreetMap avec géolocalisation des médecins proches..	28
Figure 4.4 · Profil du médecin et profil personnel.....	29
Figure 4.5 · Sélection du créneau et confirmation du rendez-vous.....	29
Figure 4.6 · Gestion des disponibilités (Manage Availability).....	30
Figure 4.7 · Dashboard du médecin et Dashboard de l'administrateur.....	31
Figure 4.8 · Gestion des disponibilités, des patients et ajout d'un médecin...	31
Figure 4.9 · Vue des rendez-vous planifiés du patient.....	32

Liste des tableaux

Tableau 1.1 — Étude comparative des solutions existantes.....	7
Tableau 1.2 — Planification des sprints de développement.....	8
Tableau 2.1 — Acteurs du système et leurs rôles.....	9
Tableau 2.2 — Besoins fonctionnels de l'application.....	10
Tableau 2.3 — Besoins non fonctionnels de l'application.....	11
Tableau 3.1 — Structure de la collection users.....	13
Tableau 3.2 — Structure de la collection doctors.....	13
Tableau 3.3 — Structure de la collection appointments.....	13
Tableau 4.1 — Environnement matériel de développement.....	15
Tableau 4.2 — Technologies utilisées dans le projet Clinica.....	16

Introduction Générale

Dans un monde où la transformation numérique révolutionne tous les secteurs, la santé n'échappe pas à cette dynamique. L'évolution rapide des technologies mobiles a profondément modifié notre rapport aux services médicaux, plaçant les applications de santé au cœur d'une révolution déterminante pour les patients et les professionnels de soin.

Malgré ces avancées technologiques, un paradoxe persiste : alors que l'information médicale n'a jamais été aussi abondante, de nombreux patients se trouvent démunis face à une question apparemment simple — comment trouver rapidement le bon professionnel de santé ? Cette difficulté s'explique par plusieurs facteurs : la dispersion des informations sur de multiples plateformes, l'absence de centralisation des données médicales, et surtout, la complexité à filtrer ces informations selon des critères pertinents tels que la spécialité, la localisation ou les disponibilités réelles.

C'est précisément pour répondre à cette problématique que l'application mobile Android « Clinica » a été développée. Conçue comme une solution centralisée et intuitive, elle ambitionne de transformer l'expérience de recherche médicale en offrant un outil capable de mettre en relation patients et professionnels de santé selon des critères précis : spécialité médicale, localisation géographique via OpenStreetMap, disponibilité en temps réel, et prise de rendez-vous directe. Au-delà de la simple mise en relation, l'application intègre une messagerie directe patient-médecin et un espace d'administration complet.

Ce rapport présente l'intégralité du travail accompli, depuis l'analyse du contexte et des besoins jusqu'à la conception de l'architecture technique, en passant par la modélisation des données Firebase, la description des interfaces réalisées sous Figma, et le plan de tests. L'approche retenue est la méthode agile Scrum, structurée en cinq sprints itératifs.

Le document s'articule en quatre chapitres principaux : l'étude préalable et la présentation du projet, l'analyse et la spécification des besoins, la conception du système, et enfin la réalisation et les interfaces de l'application.

Chapitre 1 : Étude Préalable et Présentation du Projet

Introduction — Ce premier chapitre pose les fondations du projet Clinica. Il présente le contexte institutionnel et la problématique identifiée, définit les objectifs stratégiques, analyse les solutions existantes et justifie l'approche méthodologique retenue.

1.1 Présentation du Projet

Clinica est une application mobile Android conçue pour aider les patients à trouver rapidement des professionnels de santé. L'application permet une recherche optimisée selon la spécialité médicale, la localisation géographique et les disponibilités réelles des praticiens. Elle vise à améliorer l'accès à l'information médicale grâce à une interface intuitive et adaptée aux besoins des patients tunisiens, particulièrement dans la région de Sidi Bouzid.

Le projet s'inscrit dans le cadre de la formation en Ingénierie des Applications Mobiles à l'Institut Supérieur des Études Technologiques de Sidi Bouzid (ISET Sidi Bouzid), année universitaire 2025–2026.

Nom du projet	Clinica — Optimisation de la Recherche des Professionnels de Santé
Type d'application	Application mobile native Android
Établissement	ISET Sidi Bouzid — Ingénierie des Applications Mobiles
Année universitaire	2025 - 2026
Réalisée par	Nada Nasri
Encadrant	M. Wael Bouazizi

1.2 Problématique

Malgré la disponibilité de plusieurs plateformes de recherche médicale, les utilisateurs rencontrent encore des difficultés importantes pour trouver rapidement un professionnel de santé répondant à leurs besoins spécifiques. Les informations sont souvent dispersées, incomplètes, non vérifiées, et peu adaptées aux interfaces mobiles. Cette situation entraîne une perte de temps considérable pour les patients et complique l'accès aux soins, notamment dans les régions où l'offre médicale est limitée.

L'analyse du terrain a permis d'identifier cinq obstacles majeurs :

- Données médicales dispersées sur de multiples plateformes non fiables
- Absence de géolocalisation dédiée pour trouver les médecins proches
- Interfaces non adaptées au mobile, rendant la navigation difficile
- Pas de contact direct avec le médecin avant la consultation
- Temps de recherche excessif : 20 à 30 minutes en moyenne sans résultat satisfaisant

1.3 Objectifs du Projet

L'application Clinica a été conçue pour répondre à ces défis à travers six objectifs stratégiques :

- Centralisation : regrouper les informations des professionnels de santé en une seule application

- Géolocalisation : intégrer OpenStreetMap pour localiser les médecins proches en temps réel
- Rapidité : réduire le temps de recherche à moins de 2 minutes
- Communication directe : permettre la messagerie patient-médecin avant et après la consultation
- Prise de rendez-vous : offrir un système de RDV en ligne avec sélection de créneaux
- Administration : fournir un espace de gestion des médecins et des patients pour l'administrateur

1.4 Étude de l'Existant

1.4.1 Description des solutions existantes

Avant de concevoir la solution, une analyse comparative des applications et portails existants a été conduite afin d'identifier leurs lacunes et de positionner Clinica dans l'écosystème numérique actuel.

Doctolib (France) est la plateforme de référence en Europe pour la prise de rendez-vous médicaux en ligne. Elle offre une géolocalisation et un système de RDV performants, mais son modèle économique repose sur un abonnement payant pour les médecins, ce qui limite son déploiement en Tunisie. De plus, elle ne propose pas de messagerie directe patient-médecin intégrée.

Chifaa (Algérie) est une application régionale proposant la prise de rendez-vous, mais sans géolocalisation intégrée ni messagerie. Les portails web locaux tunisiens (annuaires de médecins) sont entièrement statiques, sans application mobile native, sans système de RDV ni de suivi des disponibilités.

1.4.2 Tableau comparatif

Solution	Géoloc.	RDV	Messagerie	Mobile Natif
Doctolib (France)	✓	✓	✗	✓
Chifaa (Algérie)	✗	✓	✗	✓
Portails web locaux (TN)	✗	✗	✗	✗
Clinica (notre solution)	✓	✓	✓	✓

Tableau 1.1 — Étude comparative des solutions existantes

Clinica se distingue par sa couverture complète des quatre fonctionnalités clés — géolocalisation, prise de rendez-vous, messagerie directe et interface mobile native — absentes simultanément de toutes les solutions analysées dans la région.

1.5 Méthode de Développement Adoptée

La méthode agile Scrum a été retenue pour conduire ce projet. Elle garantit une livraison incrémentale des fonctionnalités, une adaptation rapide aux retours utilisateurs et une meilleure gestion des priorités. Le développement est planifié en cinq sprints de deux semaines chacun :

Sprint	Objectif	Livrables
Sprint 1	Authentification & Profil	Splash Screen, Inscription (Patient/Médecin), Connexion Firebase Auth, Réinitialisation mot de passe
Sprint 2	Recherche & Géolocalisation	Liste médecins, Filtrage par spécialité, Carte OpenStreetMap,

		Marqueurs interactifs
Sprint 3	Rendez-vous & Messagerie	Prise de RDV avec sélection date/créneau, Confirmation, Messagerie temps réel Firestore
Sprint 4	Interfaces Médecin & Admin	Dashboard médecin, Gestion disponibilités, Dashboard admin, Ajout/blocage médecins
Sprint 5	Tests & Finalisation	Tests UAT, Corrections, Prototype Figma complet, Déploiement APK final

Tableau 1.2 — Planification des sprints de développement

Conclusion — Ce chapitre a permis de poser le contexte et les fondations du projet Clinica. La problématique identifiée est claire : les patients tunisiens perdent un temps précieux faute d'un outil centralisé et géolocalisé pour accéder aux soins. L'étude comparative confirme la valeur ajoutée de Clinica face aux solutions existantes. Le chapitre suivant formalise les besoins fonctionnels et non fonctionnels qui guideront l'ensemble du développement.

Chapitre 2 : Analyse et Spécification des Besoins

Introduction — Ce chapitre formalise l'ensemble des exigences du système. Les acteurs sont identifiés, leurs interactions modélisées via un diagramme de cas d'utilisation, et les besoins fonctionnels et non fonctionnels sont détaillés.

2.1 Identification des Acteurs

Le système fait intervenir quatre catégories d'acteurs dont les rôles et interactions sont clairement définis :

Acteur	Rôle	Fonctionnalités principales
Patient	Utilisateur principal	Inscription, recherche de médecins, prise de RDV, messagerie, géolocalisation
Médecin	Prestataire de soins	Gestion profil, disponibilités, validation RDV, chat patient
Administrateur	Gestionnaire système	Validation profils, modération, statistiques, gestion utilisateurs
Visiteur	Utilisateur non authentifié	Consultation des profils, liste des médecins, inscription

Tableau 2.1 — Acteurs du système et leurs rôles

2.2 Diagramme de Cas d'Utilisation Global

Le diagramme de cas d'utilisation global modélise l'ensemble des interactions entre les acteurs et le système Clinica. Il distingue les cas d'utilisation disponibles pour chaque type d'acteur :

- Visiteur : consulter la liste des médecins, voir les profils, s'inscrire
- Patient : rechercher un médecin, prendre un RDV, envoyer un message, géolocaliser
- Médecin : gérer son profil, définir ses disponibilités, valider les RDV, communiquer
- Administrateur : ajouter/bloquer des médecins, gérer les patients, consulter les statistiques

2.3 Besoins Fonctionnels

Les sept modules fonctionnels identifiés couvrent le cycle complet de la relation patient-professionnel de santé :

ID	Module	Description
BF-01	Authentification	Inscription, connexion email/mot de passe via Firebase Auth, réinitialisation mot de passe
BF-02	Recherche de médecins	Filtrage par spécialité, localisation, disponibilité — résultats triés par distance
BF-03	Géolocalisation	Carte OpenStreetMap interactive, marqueurs des médecins disponibles/indisponibles

Figure 2.1 — Diagramme de Cas d'Utilisation : Médecin-Patient

Le diagramme Médecin-Patient modélise les interactions principales au sein du système Clinica. Le Médecin gère son profil, sa messagerie (envoi, modification, suppression, blocage), ses rendez-vous et ses disponibilités (jours/heures de travail). Le Patient interagit pour planifier un rendez-vous et communiquer via la messagerie. Les deux acteurs s'authentifient et remplissent des formulaires pour toute action de modification (relations <<Include>>).

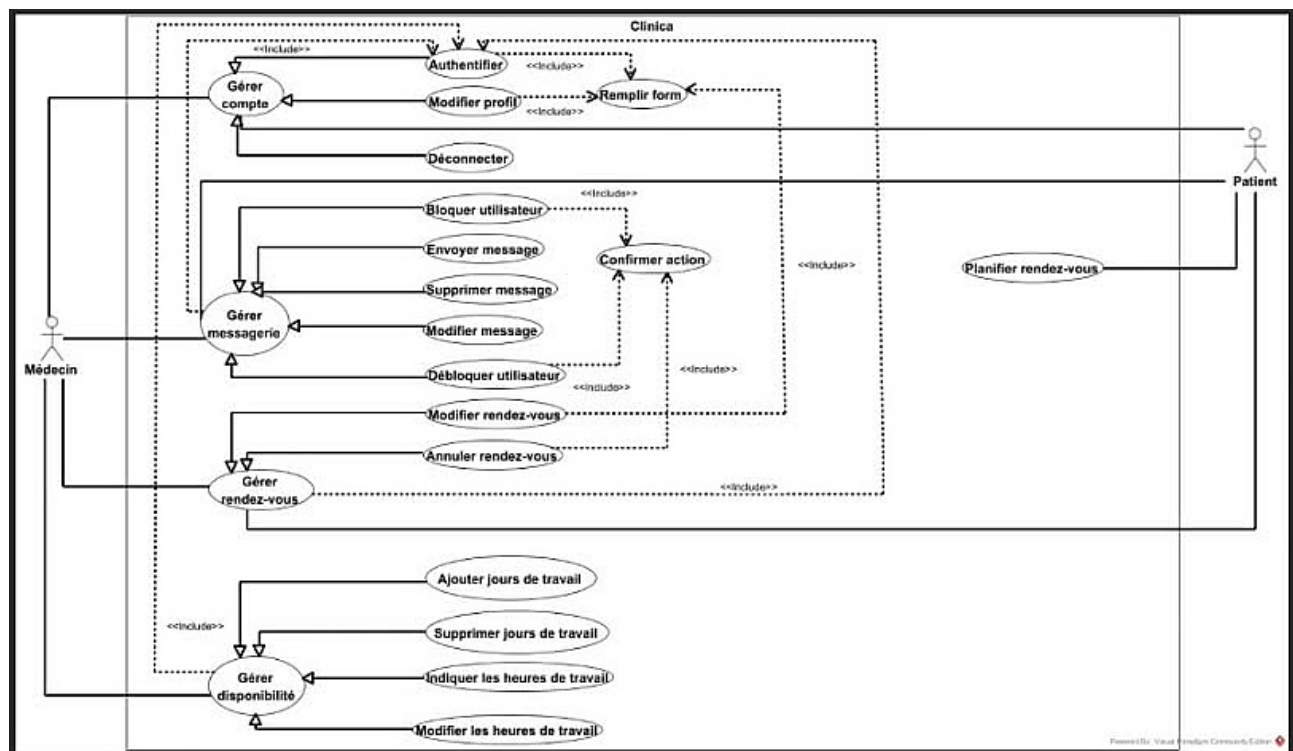


Figure 2.1 — Diagramme de Cas d'Utilisation Global — Interactions Médecin-Patient

Figure 2.2 — Diagramme de Cas d'Utilisation : Administrateur

L'Administrateur gère les sessions (authentification/déconnexion) via Gérer compte.
Il supervise les utilisateurs : ajout, modification et blocage/déblocage de médecins.
Les actions critiques (ajout médecin, blocage) dépendent d'étapes secondaires :
remplir un formulaire ou confirmer l'action (relations <<Include>>).

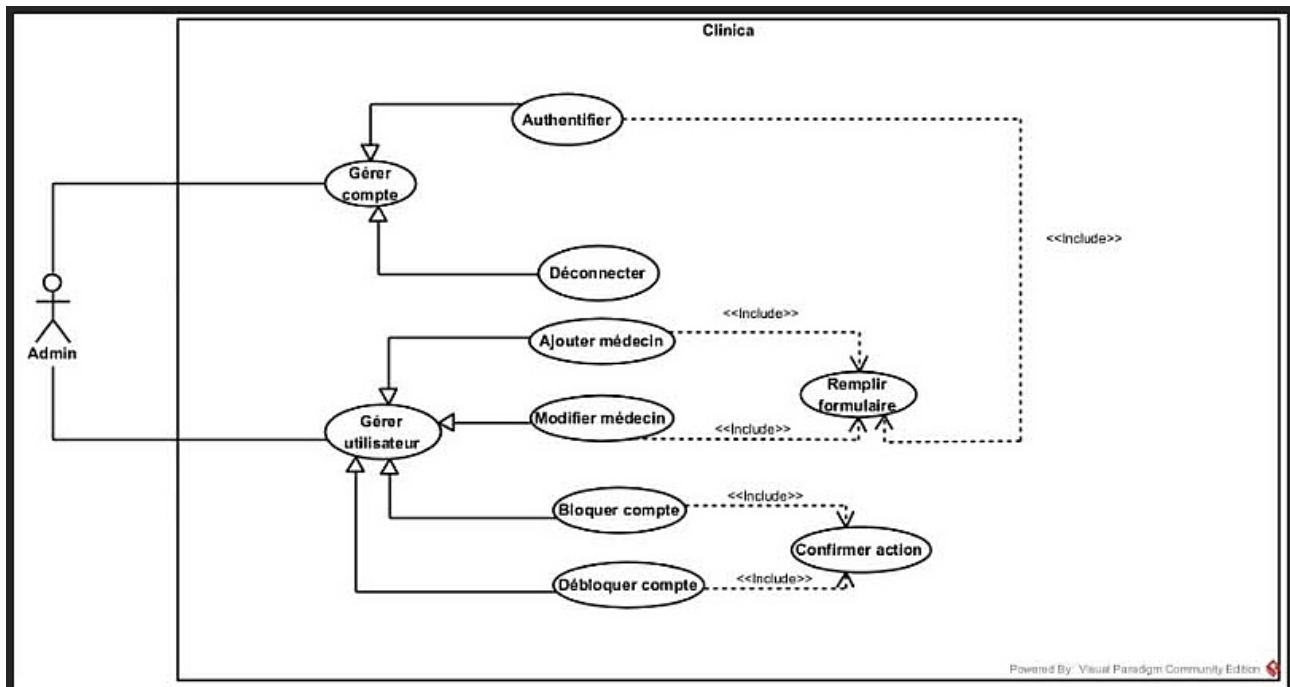


Figure 2.2 — Diagramme de Cas d'Utilisation — Rôle Administrateur

BF-04	Prise de rendez-vous	Sélection date + créneau horaire, confirmation enregistrée dans Firebase Firestore
BF-05	Messagerie patient-médecin	Échange de messages en temps réel via Firebase Firestore
BF-06	Gestion des profils	Médecin : profil complet avec photo, spécialité, bio, coordonnées
BF-07	Administration	Ajout/blocage de médecins, gestion des patients, modération

Tableau 2.2 — Besoins fonctionnels de l'application

2.3.1 Authentification sécurisée

Le module d'authentification est le point d'entrée de l'application. Il comprend l'inscription avec validation complète (nom, email, téléphone, mot de passe sécurisé), la connexion via Firebase Auth avec gestion des erreurs, et la récupération de mot de passe par email. Une redirection automatique vers l'espace approprié (patient, médecin ou administrateur) est assurée selon le rôle enregistré dans Firestore.

2.3.2 Recherche et Géolocalisation

La recherche de médecins permet de filtrer par spécialité médicale (cardiologie, pédiatrie, neurologie, médecine générale, ophtalmologie, orthopédie) et d'afficher les résultats avec la distance depuis la position de l'utilisateur. La carte OpenStreetMap affiche les marqueurs interactifs des médecins disponibles, permettant un accès rapide à leur profil et à la prise de rendez-vous.

2.3.3 Prise de rendez-vous

Le système de rendez-vous permet au patient de sélectionner un médecin, de choisir une date parmi les créneaux disponibles (définis par le médecin) et de confirmer un horaire précis. La confirmation est enregistrée dans Firebase Firestore avec une référence unique. Le médecin reçoit la notification et peut valider ou reprogrammer le rendez-vous depuis son tableau de bord.

2.3.4 Messagerie patient-médecin

La messagerie directe permet l'échange de messages en temps réel entre patients et médecins via Firebase Firestore. Elle offre une liste des conversations actives avec le dernier message et la date, et un fil de messages chronologique dans chaque conversation.

2.4 Besoins Non Fonctionnels

Critère	Description
Performance	Chargement des écrans < 2 secondes sur connexion 3G/4G ; réponses Firebase < 500 ms
Sécurité	Chiffrement TLS, règles de sécurité Firestore par rôle, authentification Firebase avec tokens JWT
Disponibilité	Service disponible 24h/24, 7j/7 grâce à l'infrastructure Google Cloud (SLA 99,95 %)
Compatibilité	Application Android (API 21+), interface responsive adaptée aux différentes tailles d'écran

Maintenabilité	Architecture en couches, code Kotlin modulaire, séparation Model/View/ViewModel
Ergonomie	Interface intuitive, navigation simple, design cohérent respectant les standards Material Design

Tableau 2.3 — Besoins non fonctionnels de l'application

Conclusion — Ce chapitre a formalisé l'ensemble des exigences du système. Les quatre acteurs identifiés, les sept modules fonctionnels et les six critères non fonctionnels constituent le référentiel qui guidera l'ensemble du développement. Le chapitre suivant détaille l'architecture technique retenue pour implémenter ces fonctionnalités.

Chapitre 3 : Conception du Système

Introduction — Ce chapitre présente les choix architecturaux, la modélisation des données Firestore, les diagrammes UML de conception et le prototype Figma de l'application Clinica.

3.1 Architecture du Système

3.1.1 Vue d'ensemble

L'architecture de Clinica repose sur le paradigme cloud-first mobile : l'application Kotlin native constitue la couche de présentation, Firebase assure l'intégralité du backend, et les interactions entre les deux niveaux sont sécurisées par des communications HTTPS et des règles de sécurité basées sur les rôles. Cette architecture élimine le besoin d'un serveur backend dédié, réduisant les coûts d'infrastructure tout en garantissant une scalabilité automatique.

3.1.2 Architecture en quatre couches

L'architecture physique s'articule autour de quatre niveaux interconnectés :

- Couche Client : le smartphone Android exécutant l'application Kotlin. Elle gère l'interface utilisateur native avec une architecture MVVM (Model-View-ViewModel) et communique avec Firebase via des requêtes HTTPS sécurisées.
- Couche Services / API : Firebase Auth (authentification), Firestore (base de données NoSQL temps réel), Firebase Storage (photos et documents), FCM (notifications push), et OpenStreetMap via le SDK Osmroid pour la cartographie.
- Couche Données : Cloud Firestore organisé en collections structurées — users, doctors, appointments, conversations, messages — avec synchronisation en temps réel native.

3.2 Modélisation des Données Firebase Firestore

L'application utilise Cloud Firestore comme base de données NoSQL orientée documents. Ce modèle offre une synchronisation en temps réel native, une scalabilité automatique gérée par Google Cloud, et des requêtes flexibles sur les attributs des documents.

3.2.1 Collection users

Champ	Type	Description
uid	String	Identifiant unique issu de Firebase Authentication
name	String	Nom complet de l'utilisateur
email	String	Adresse e-mail (identifiant de connexion)
phone	String	Numéro de téléphone
role	String	Rôle : 'patient', 'doctor' ou 'admin'
avatarUrl	String?	URL de la photo de profil
createdAt	Timestamp	Date et heure de création du compte

Tableau 3.1 — Structure de la collection users

3.2.2 Collection doctors

Champ	Type	Description
uid	String	Référence à users.uid
name	String	Nom complet du médecin
specialty	String	Spécialité médicale (Cardiologie, Neurologie, etc.)
bio	String	Description et parcours du médecin
location	GeoPoint	Coordonnées GPS du cabinet
phone	String	Numéro de contact
isVerified	Boolean	Profil validé par l'administrateur
isBlocked	Boolean	Compte bloqué par l'administrateur
availability	Map[]	Liste des jours et créneaux horaires disponibles

Tableau 3.2 — Structure de la collection doctors

3.2.3 Collection appointments

Champ	Type	Description
id	String	Identifiant unique du rendez-vous
patientId	String	Référence à users.uid (patient)
doctorId	String	Référence à doctors.uid
appointmentDate	Timestamp	Date et heure du rendez-vous
reason	String	Motif de la consultation
status	String	État : scheduled completed cancelled

Tableau 3.3 — Structure de la collection appointments

3.3 Diagrammes de Conception

3.3.1 Diagramme de Classes

Le diagramme de classes modélise la structure statique du système. La classe centrale est Utilisateur, dont héritent trois sous-classes spécialisées : Patient, Médecin et Administrateur. La classe RendezVous associe un Patient à un Médecin avec des détails sur la date, l'heure et le statut. La classe Conversation lie un Patient et un Médecin et contient une collection de Messages. L'Administrateur dispose des méthodes de gestion des utilisateurs et de modération du système.

3.3.2 Diagrammes de Séquence

Les principaux diagrammes de séquence décrivent les flux d'interaction critiques du système :

- Authentification : l'utilisateur saisit ses identifiants → FirebaseAuth vérifie les credentials → Firestore retourne le rôle → redirection vers le Dashboard approprié (Patient / Médecin / Admin).
- Prise de rendez-vous : le patient sélectionne un médecin → consulte les créneaux disponibles dans Firestore → choisit date et heure → le rendez-vous est enregistré dans la collection appointments avec le statut 'scheduled'.

Figure 3.1 — Architecture Générale du Système

L'architecture repose sur quatre couches : Présentation (Interface Utilisateur Android/Kotlin), Logique Métier (Serveur/API Firebase), Base de Données (Firestore — synchronisation temps réel), et Services Externes (OpenStreetMap API pour la cartographie géolocalisée).

Les couches communiquent via requêtes HTTP sécurisées, assurant séparation des responsabilités et scalabilité automatique via Google Cloud.

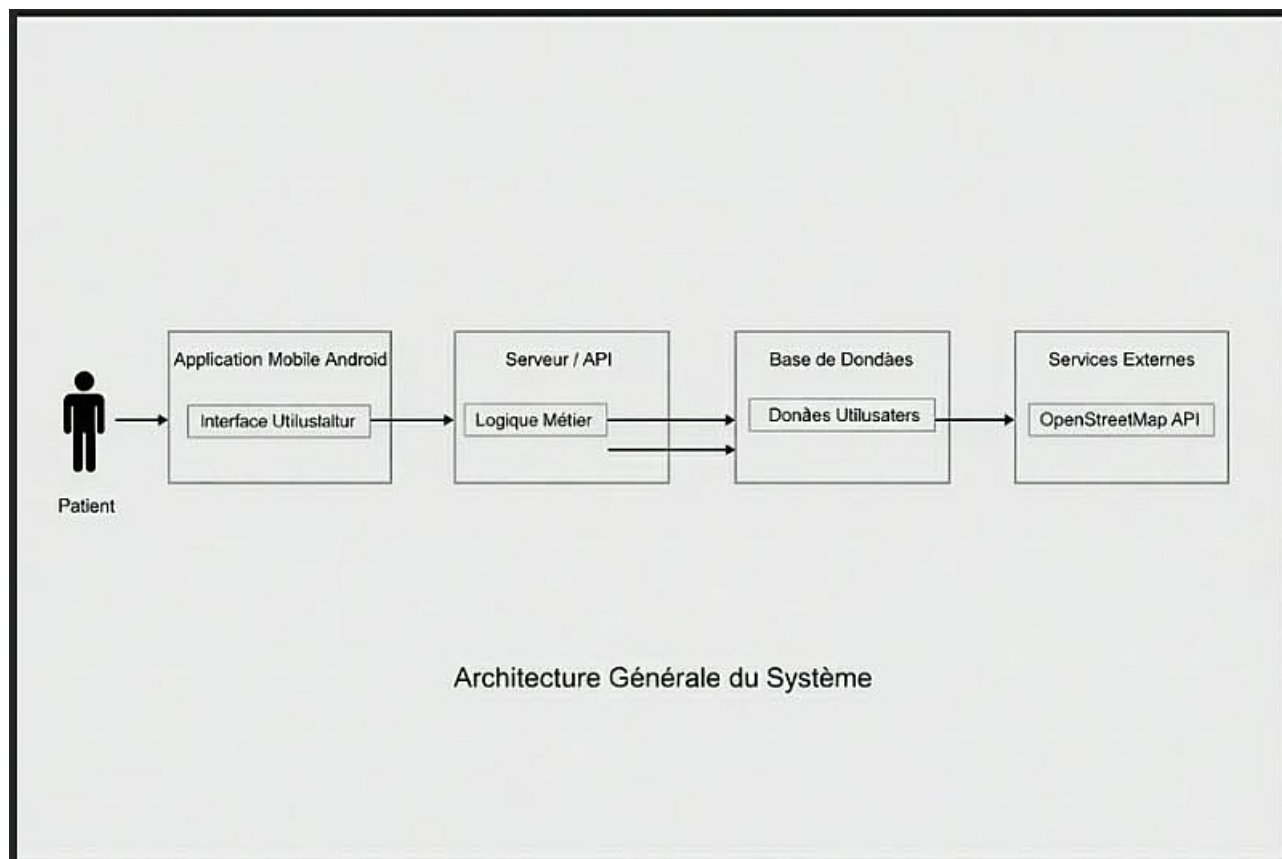


Figure 3.1 — Architecture Générale du Système Clinica

Figure 3.2 — Diagramme de Classes

La classe centrale Utilisateur est héritée par Patient, Médecin et Administrateur.

RendezVous associe Patient et Médecin (date, heure, statut). Conversation lie Patient et Médecin et contient une collection de Messages (senderId, text, timestamp).

L'Administrateur dispose de bloquerUtilisateur(), debloquerUtilisateur() et gererUtilisateurs().

Les multiplicités UML (1, 1..*, 0..*) définissent les cardinalités entre toutes les classes.

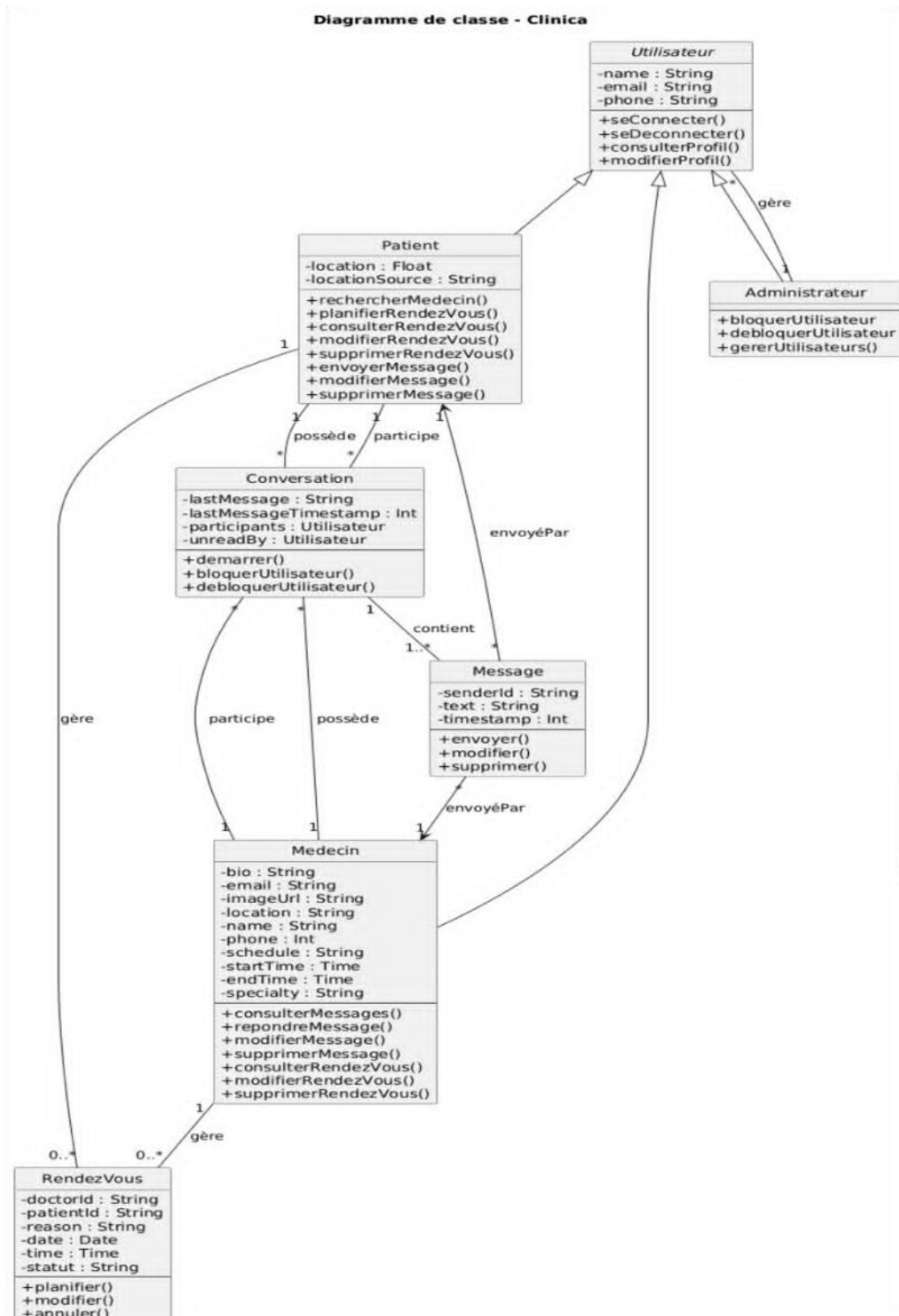


Figure 3.2 — Diagramme de Classes — Clinica

Figure 3.3 — Diagramme de Séquence : Connexion utilisateur

L'utilisateur remplit le formulaire de connexion sur LoginScreen.

LoginViewModel appelle login() → FirebaseAuth vérifie les credentials.

[Connexion réussie] : Firestore retourne le rôle (Admin/Médecin/Patient)

→ navigation automatique vers le bon Dashboard.

[Erreur] : un message d'erreur est retourné et affiché à l'utilisateur.

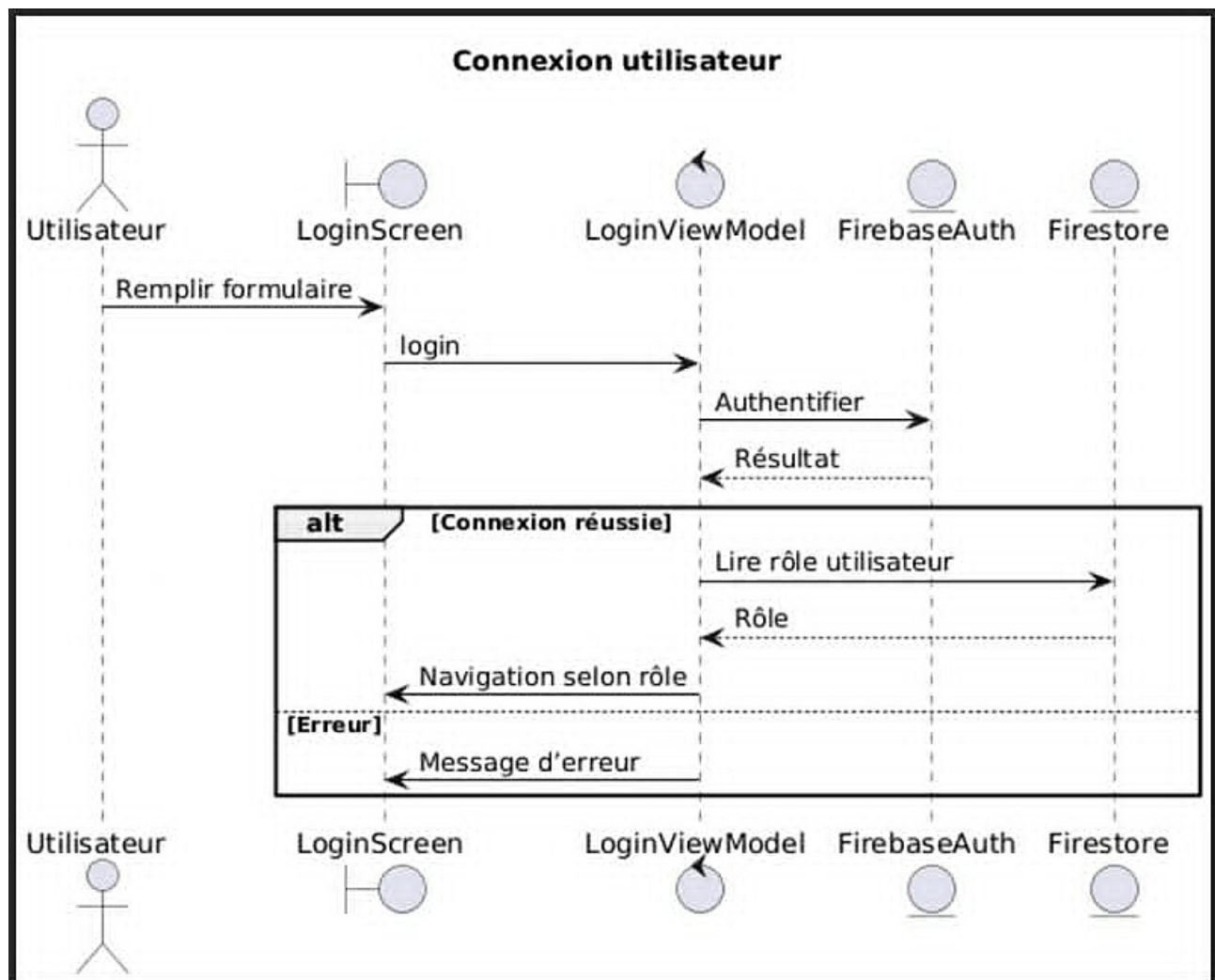


Figure 3.3 — Diagramme de Séquence — Authentification (Login)

Figure 3.4 — Diagramme de Séquence : Messagerie Patient-Médecin

Envoi message : si la conversation est active, envoyerMessage() ajoute le message dans Firestore et met à jour la conversation. Si bloquée, l'envoi est désactivé.

Blocage/Déblocage : changerEtatBlocage() met à jour l'état dans Firestore en temps réel, puis active ou désactive le champ de saisie côté client.

Modifier/Supprimer message : Editer/Supprimer déclenche une mise à jour Firestore suivie d'un rafraîchissement automatique des messages.

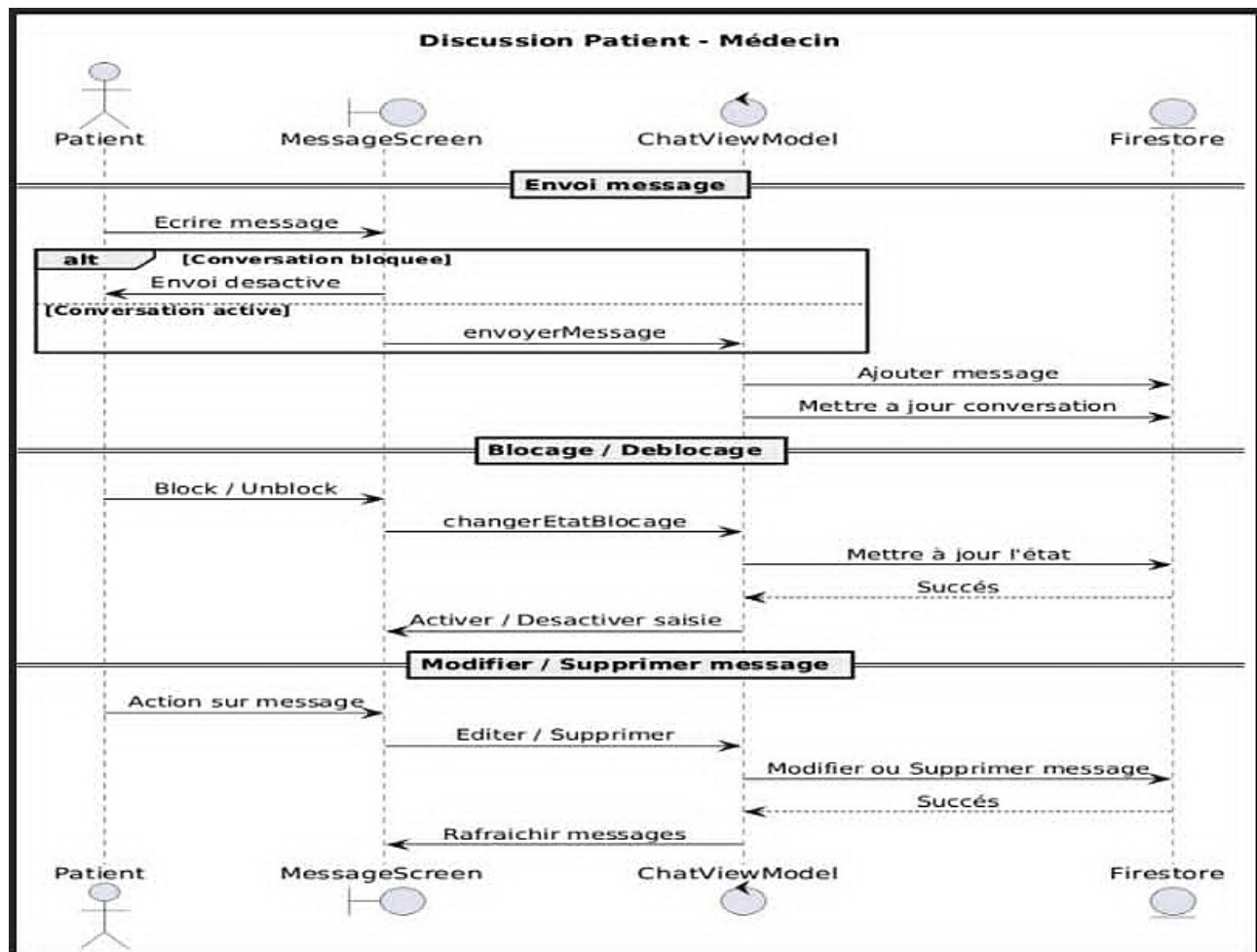


Figure 3.4 — Diagramme de Séquence — Discussion Patient-Médecin

Figure 3.5 — Diagramme de Séquence : Prise de Rendez-vous

Le Patient choisit une date et une heure sur ScheduleScreen.

AppointmentViewModel appelle creerRendezVous() → vérifie dans Firestore si le créneau est libre.

[Rendez-vous déjà existant] : message 'Rendez-vous déjà pris' affiché au Patient.

[Aucun rendez-vous] : le rendez-vous est enregistré dans Firestore avec succès.

[Erreur] : un message d'erreur générique est retourné à l'interface.

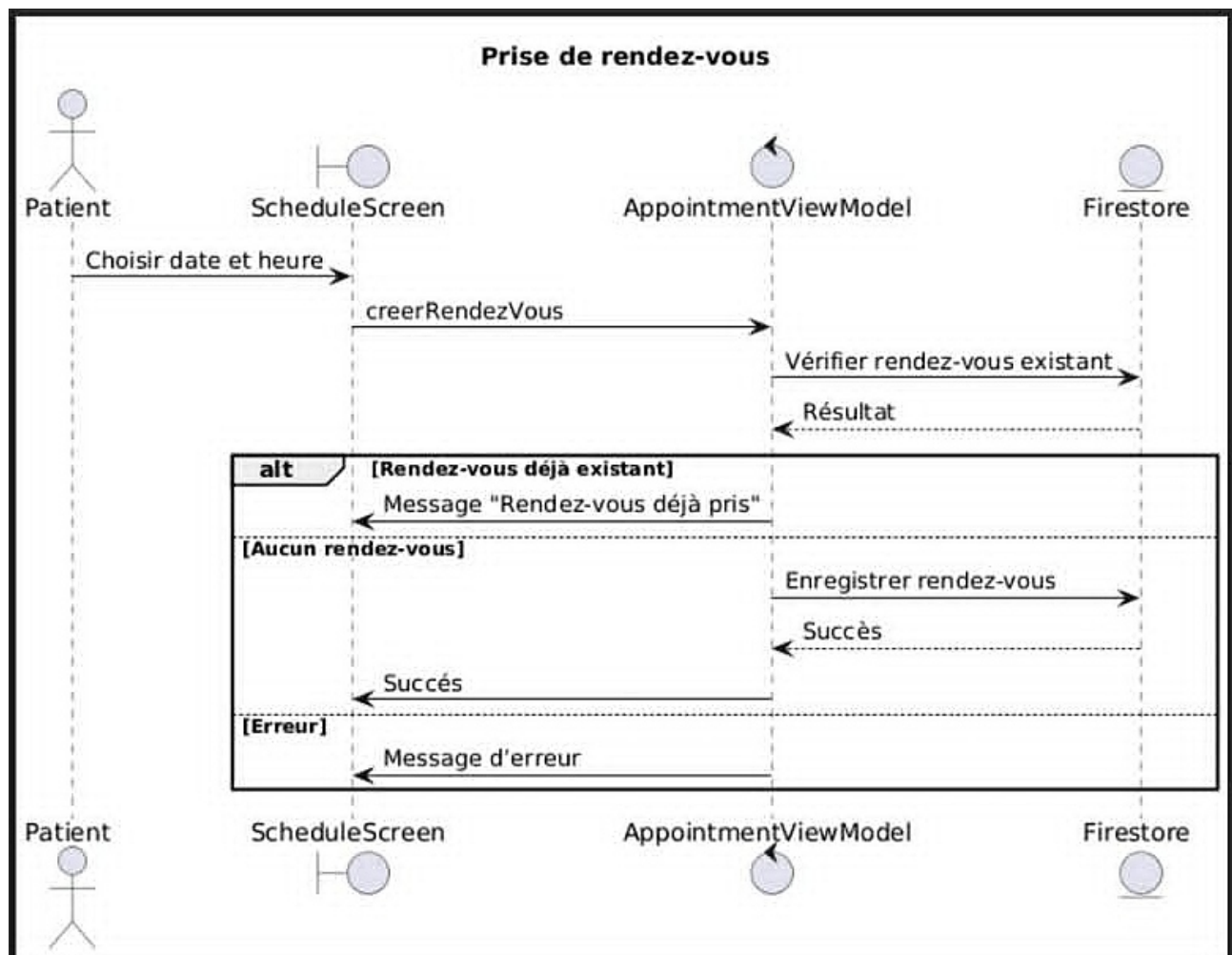


Figure 3.5 — Diagramme de Séquence — Planifier un Rendez-vous

Figure 3.6 — Diagramme de Séquence : Ajout d'un Médecin (Admin)

L'Administrateur remplit le formulaire AddDoctorScreen.

SignupViewModel appelle creerMedecin() → crée le compte dans FirebaseAuth.

[Création réussie] : les données du médecin sont enregistrées dans Firestore et un email de réinitialisation de mot de passe est envoyé automatiquement.

[Erreur] : un message d'erreur est affiché à l'Administrateur.

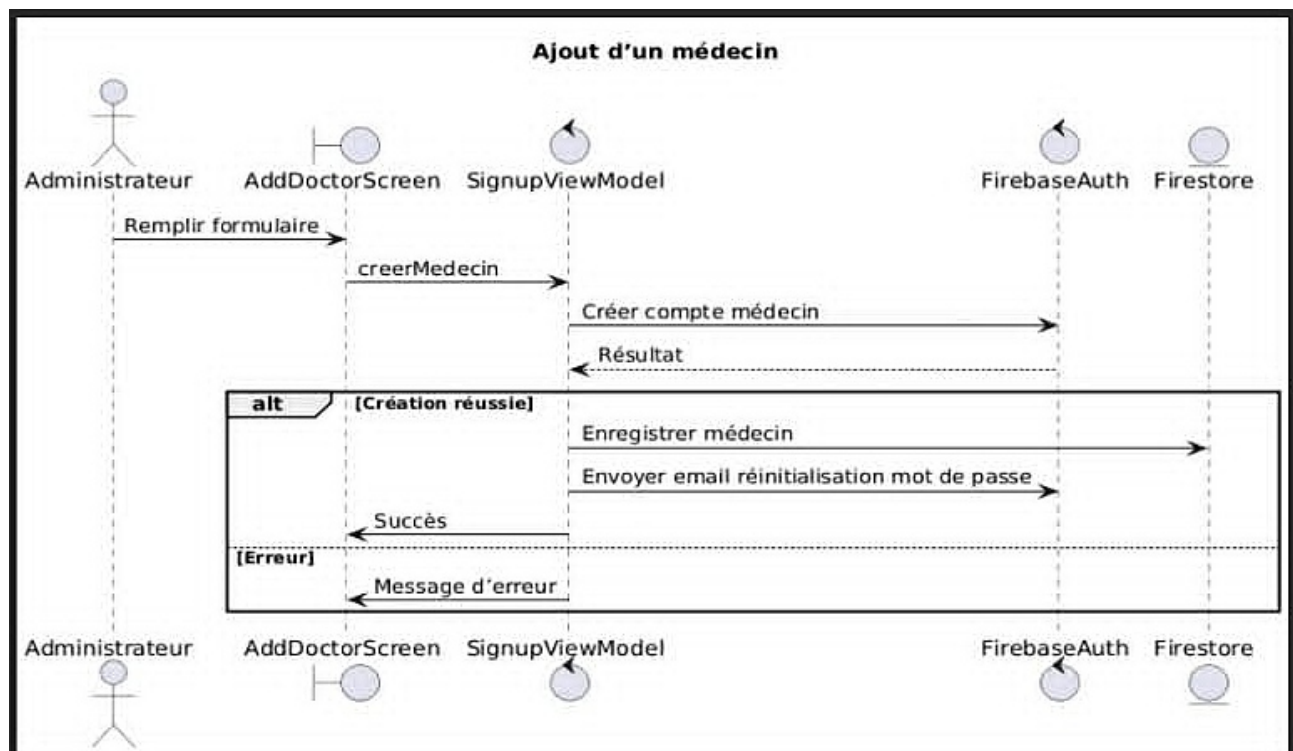


Figure 3.6 — Diagramme de Séquence — Ajout d'un Médecin par l'Administrateur

-
- Messagerie : l'utilisateur ouvre une conversation → les messages sont chargés depuis Firestore en temps réel → l'envoi crée un nouveau document dans la sous-collection messages avec horodatage.
 - Ajout de médecin (Admin) : l'admin remplit le formulaire → le système crée le compte dans FirebaseAuth → enregistre les données dans Firestore → envoie un email de bienvenue au nouveau médecin.

3.4 Prototype Figma

Un prototype interactif haute-fidélité a été conçu sur Figma pour l'intégralité des écrans de l'application. Ce prototype a permis de valider l'ergonomie et la cohérence visuelle avant le développement, réduisant ainsi les corrections post-implémentation.

Le Design System Figma comprend : la palette de couleurs (gradient bleu-teal comme couleur principale, blanc pour les surfaces, gris clair pour les fonds), la typographie (Roboto pour les contenus, poids variable selon la hiérarchie), les composants réutilisables (boutons, cartes médecin, champs de saisie, badges de statut) et la grille d'interface (marges de 16dp, espacement 8dp).

Conclusion — Ce chapitre a présenté l'architecture technique complète de Clinica. Le choix de Firebase comme backend cloud garantit la scalabilité et la disponibilité, tandis que l'architecture MVVM en Kotlin assure la maintenabilité du code. Le prototype Figma a constitué un outil de validation précieux tout au long des sprints Scrum. Le chapitre suivant présente les interfaces réalisées et les technologies utilisées.

Chapitre 4 : Réalisation et Interfaces de l'Application

Introduction — Ce chapitre présente l'environnement de développement, les technologies utilisées et l'ensemble des interfaces réalisées, illustrées par les captures d'écran réelles de l'application.

4.1 Environnement de Développement

4.1.1 Environnement matériel

Caractéristique	Spécification
Marque	MSI
Processeur (CPU)	12th Gen Intel Core i7-12650H — 2,30 GHz
Carte graphique (GPU)	NVIDIA RTX 3050
Mémoire RAM	32 Go
Stockage	512 Go SSD NVMe
Système d'exploitation	Microsoft Windows 11 Professionnel

Tableau 4.1 — Environnement matériel de développement

4.1.2 Environnement logiciel

Les outils suivants ont été utilisés pour le développement et la conception de l'application :

- Android Studio (Hedgehog 2023.1.1) : IDE officiel Android, support natif Kotlin, Layout Editor, émulateurs intégrés, débogage avancé et gestionnaire de dépendances Gradle.
- Figma : outil de conception UI/UX pour les maquettes haute-fidélité, le prototypage interactif et le Design System de l'application.
- Visual Paradigm : création des diagrammes UML (cas d'utilisation, séquences, classes).
- Git / GitHub : gestion du versionnement du code source avec historique des commits par sprint.

4.2 Technologies Utilisées

Technologie	Domaine d'utilisation	Justification
Kotlin	Langage principal Android	Concision, sécurité des types nuls, interopérabilité Java
Firebase Auth	Authentification	Gestion des identités, tokens JWT, réinitialisation email
Firebase Firestore	Base de données NoSQL	Synchronisation temps réel, requêtes flexibles, scalabilité
Firebase Storage	Stockage fichiers	Photos de profil, documents, contrôle d'accès par règles
OpenStreetMap	Cartographie & géoloc.	Cartes interactives, marqueurs médecins, localisation GPS

ImgBB	Hébergement images	API REST simple, upload photos de profil, URL permanentes
Figma	Conception UI/UX	Maquettes haute-fidélité, prototype interactif, design system
Android Studio	IDE de développement	Support natif Kotlin, émulateurs, débogage avancé, Gradle

Tableau 4.2 — Technologies utilisées dans le projet Clinica

4.2.1 Kotlin — Langage natif Android

Kotlin a été choisi comme langage principal pour sa concision, sa sécurité (gestion native des types nuls évitant les `NullPointerException`), son interopérabilité avec Java et son support complet par Android Studio. Les coroutines Kotlin ont été utilisées pour la gestion asynchrone des appels Firebase, assurant une interface toujours réactive.

4.2.2 Firebase — Backend as a Service

Firebase constitue l'épine dorsale du backend de l'application. Firebase Auth assure la gestion sécurisée des identités avec tokens JWT et gestion des rôles. Cloud Firestore fournit une base de données NoSQL avec synchronisation en temps réel, idéale pour les mises à jour instantanées des disponibilités et des messages. Firebase Storage assure l'hébergement des photos de profil. Les règles de sécurité Firestore garantissent que chaque utilisateur n'accède qu'aux données correspondant à son rôle.

4.2.3 OpenStreetMap — Cartographie libre

OpenStreetMap a été intégré via la bibliothèque `Osmdroid` pour Android, offrant des cartes interactives open-source sans quota d'utilisation ni facturation à la requête. Les marqueurs personnalisés distinguent visuellement les médecins disponibles (vert) des médecins indisponibles, et un champ de recherche par spécialité filtre les résultats affichés sur la carte.

4.3 Présentation des Interfaces de l'Application

4.3.1 Écrans d'authentification

Les écrans d'entrée de l'application (splash screen, inscription, connexion) respectent la charte graphique de Clinica : fond en dégradé bleu-teal, typographie blanche, boutons arrondis. L'écran de création de compte collecte le nom complet, l'adresse email, le numéro de téléphone et le mot de passe sécurisé.

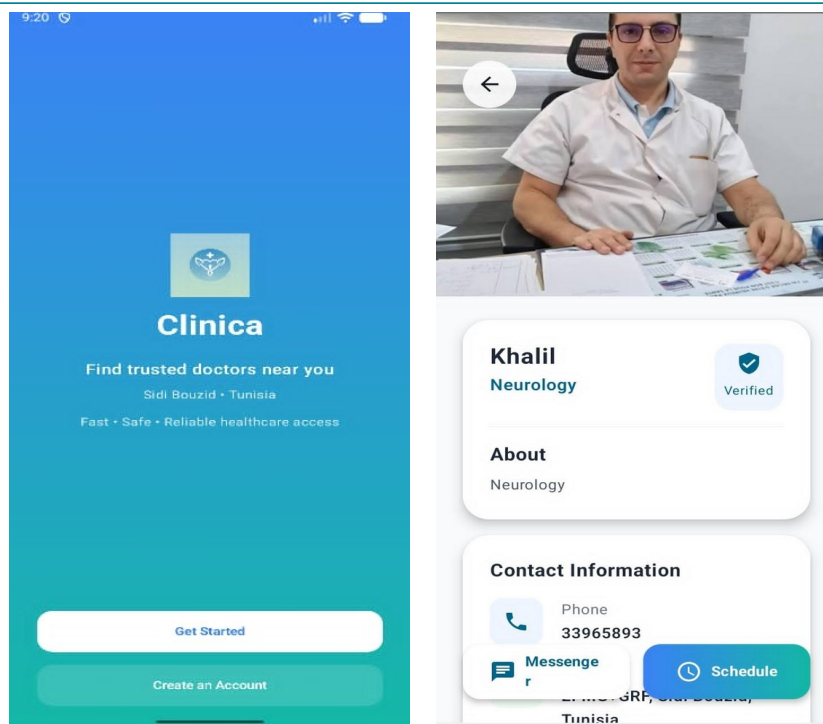


Figure 4.1 — Écran d'accueil (Splash Screen) et création de compte

4.3.2 Écran des spécialités et recherche

L'écran des spécialités présente les six spécialités disponibles (Cardiologie, Pédiatrie, Neurologie, Médecine générale, Ophtalmologie, Orthopédie) sous forme de grille avec icônes colorées. La sélection d'une spécialité charge la liste des médecins correspondants avec leur distance.

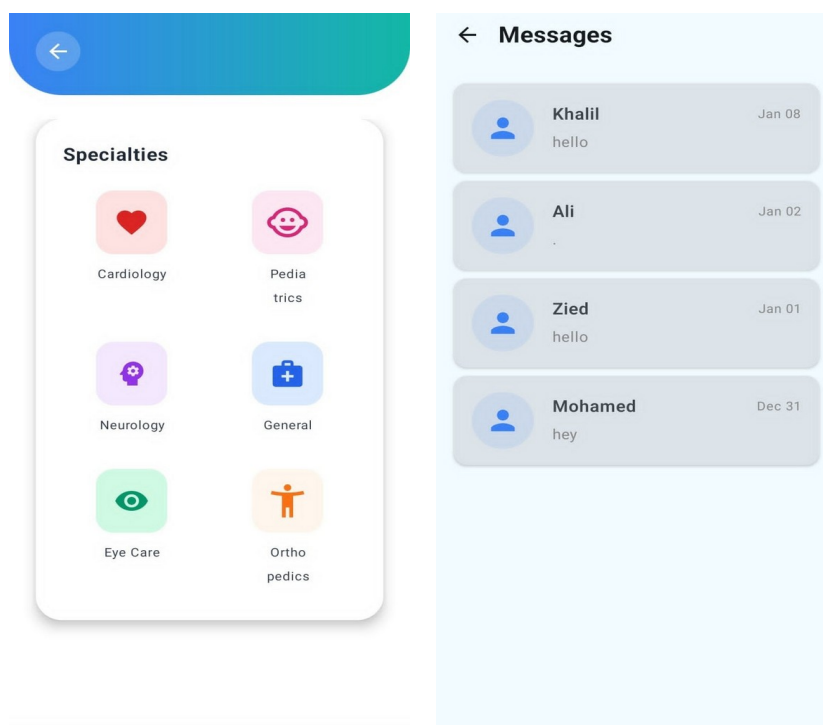


Figure 4.2 — Écrans des spécialités et liste des médecins

4.3.3 Carte de géolocalisation

L'écran de géolocalisation affiche une carte OpenStreetMap centrée sur la position de l'utilisateur avec les marqueurs des médecins proches. Un champ de recherche par spécialité filtre les résultats en temps réel. Le tap sur un marqueur affiche un aperçu du profil du médecin avec un accès direct à la prise de rendez-vous.

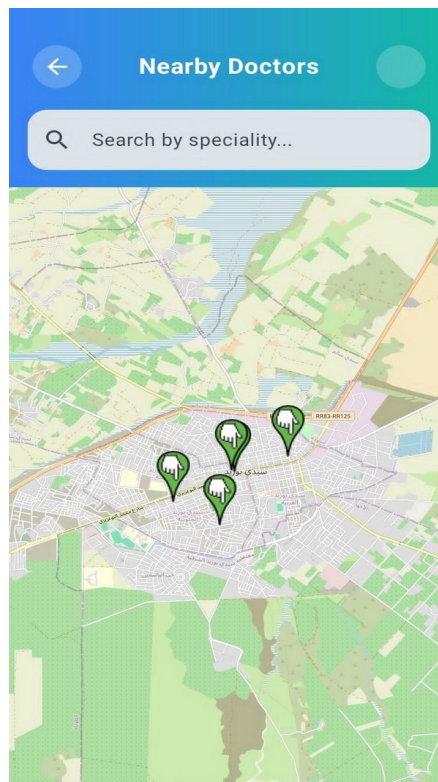


Figure 4.3 — Carte OpenStreetMap avec géolocalisation des médecins proches

4.3.4 Profil du médecin

La page de profil du médecin affiche sa photo, son nom, sa spécialité et un badge de vérification. La section « À propos » présente sa biographie. La section « Contact » affiche son numéro de téléphone et un bouton « Messenger » pour ouvrir la messagerie directe. Le bouton « Schedule » en bas permet d'initier la prise de rendez-vous.

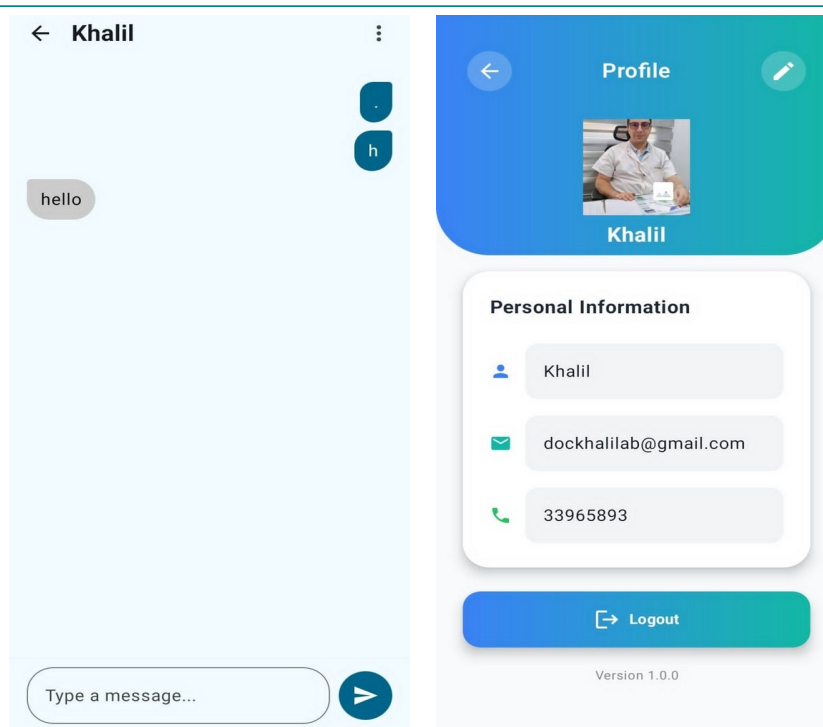


Figure 4.4 — Profil du médecin et profil personnel

4.3.5 Prise de rendez-vous et confirmation

L'écran de sélection de rendez-vous présente les dates disponibles sous forme de grille cliquable (créneaux définis par le médecin) et les horaires correspondants. Une fois la date et l'heure sélectionnées, le bouton « Confirm Appointment » enregistre le rendez-vous dans Firebase et affiche l'écran de confirmation avec les détails complets.

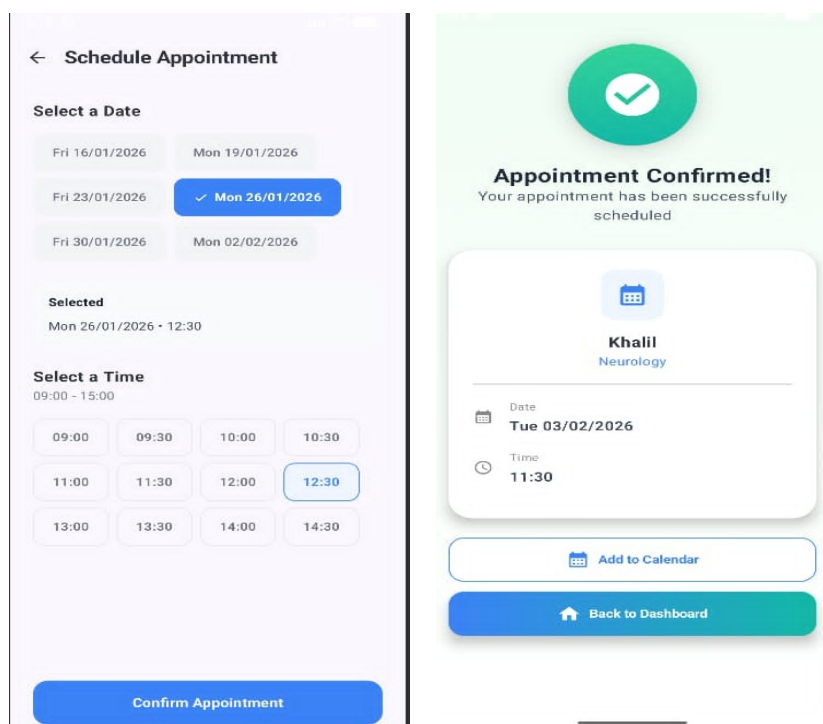


Figure 4.5 — Sélection du créneau et confirmation du rendez-vous

4.3.6 Messagerie

L'écran des messages affiche la liste des conversations actives avec le nom du correspondant, le dernier message et la date. L'écran de conversation présente le fil de messages chronologique avec distinction visuelle entre les messages envoyés (bleu teal, alignés à droite) et reçus (gris clair, alignés à gauche). Un champ de saisie avec bouton d'envoi est toujours visible en bas de l'écran.

← **Manage Availability**

Set Your Working Schedule
Select days and set individual hours for each

+ Add Working Day

Wednesday ✕

Start Time End Time

09:00 13:00

Tuesday ✕

Start Time End Time

09:00 13:00

Register Schedule

Figure 4.6 · Gestion des disponibilités (Manage Availability)

4.3.7 Tableaux de bord Médecin et Administrateur

Le tableau de bord du médecin affiche le prochain rendez-vous planifié, un accès rapide à la gestion des disponibilités (Set Availability) et la liste des rendez-vous à venir avec les boutons Reprogrammer / Annuler. Le tableau de bord de l'administrateur propose les actions rapides « Add New Doctor » et deux sections de gestion : Manage Doctors et Manage Patients.

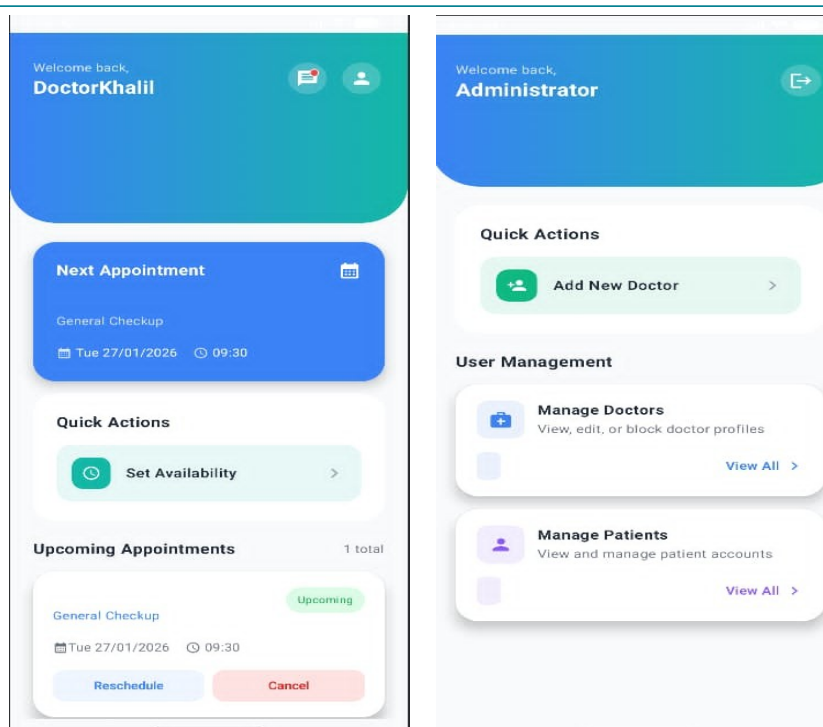


Figure 4.7 — Dashboard du médecin et Dashboard de l'administrateur

4.3.8 Gestion des disponibilités et des patients (Admin/Médecin)

L'écran de gestion des disponibilités (Manage Availability) permet au médecin de définir ses jours et plages horaires de travail en ajoutant des journées avec les heures de début et de fin. L'écran de gestion des patients (Manage Patients) affiche la liste complète avec statut (actif ou bloqué) et le bouton de blocage/déblocage pour l'administrateur. L'écran Add New Doctor permet à l'administrateur d'ajouter un nouveau praticien avec photo, nom, spécialité, email, bio et localisation sur carte.

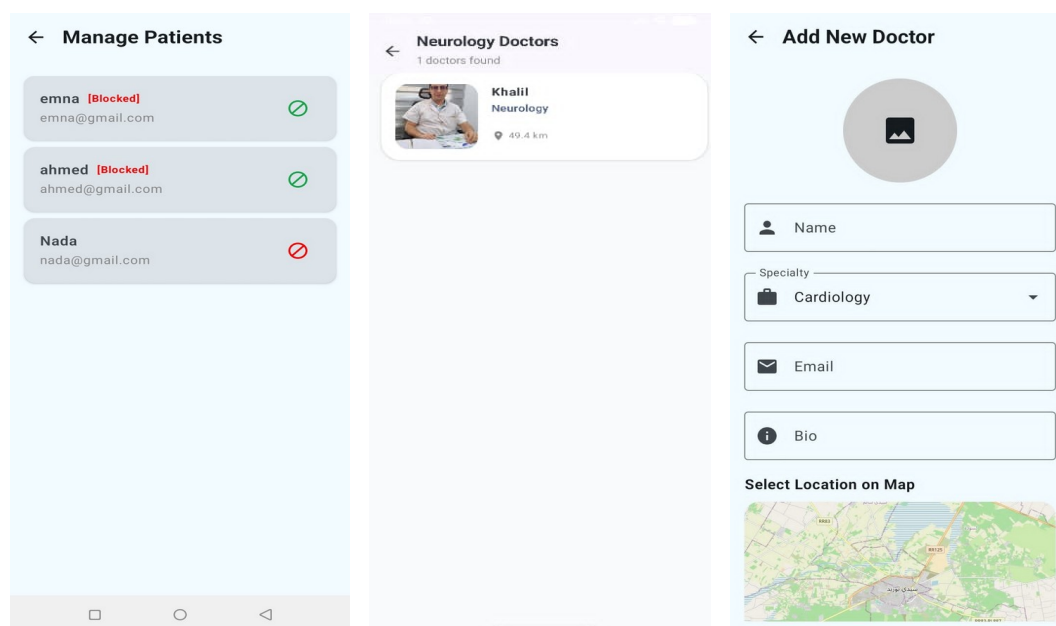


Figure 4.8 — Gestion des disponibilités, des patients et ajout d'un médecin

4.3.9 Rendez-vous du patient

L'écran des rendez-vous du patient (onglet Appointments) affiche la liste des rendez-vous à venir avec le nom du médecin, la spécialité, la date, l'heure et le statut (Upcoming / Completed / Cancelled). Chaque carte propose les actions Reprogrammer et Annuler.

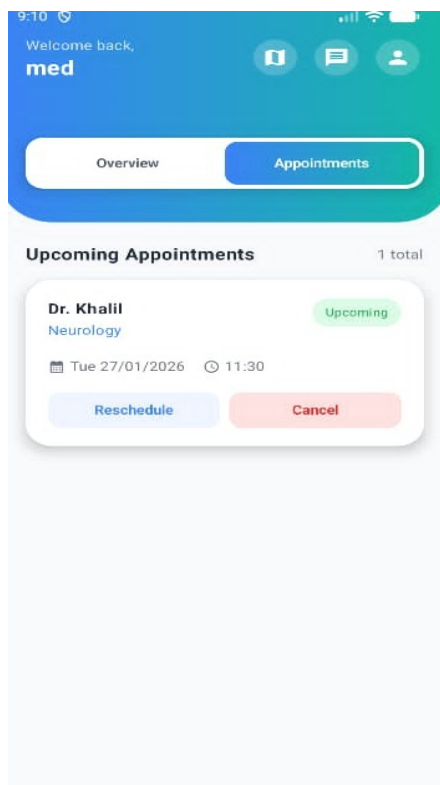


Figure 4.9 — Vue des rendez-vous planifiés du patient

Conclusion — Ce chapitre a présenté l'ensemble des interfaces réalisées de l'application Clinica, de l'écran d'accueil jusqu'aux espaces d'administration avancés. Les 17 écrans développés couvrent intégralement les besoins fonctionnels identifiés au chapitre 2. L'intégration de Firebase garantit des mises à jour en temps réel, tandis qu'OpenStreetMap offre une expérience de géolocalisation fluide sans contrainte de quota.

Conclusion Générale

Ce rapport a présenté l'intégralité du projet Clinica, une application mobile Android visant à moderniser et à simplifier l'accès aux professionnels de santé pour les patients tunisiens, particulièrement dans la région de Sidi Bouzid.

L'analyse de l'existant (Chapitre 1) a mis en évidence les lacunes profondes des solutions actuelles — portails web statiques, absence de géolocalisation dédiée, temps de recherche excessif — et a justifié la conception d'une solution mobile native. La spécification des besoins (Chapitre 2) a formalisé sept modules fonctionnels couvrant le cycle complet de la relation patient-médecin. L'architecture technique (Chapitre 3), reposant sur Kotlin et Firebase, garantit performance, sécurité et disponibilité. La réalisation (Chapitre 4) présente les dix-sept interfaces développées, illustrant la concrétisation de chaque besoin fonctionnel identifié.

La solution Clinica se distingue par plusieurs apports majeurs :

- Une géolocalisation complète via OpenStreetMap permettant de trouver les médecins proches en temps réel
- Un système de rendez-vous en ligne avec sélection de créneaux disponibles définis par les médecins
- Une messagerie directe patient-médecin fonctionnant en temps réel grâce à Firebase Firestore
- Un espace d'administration complet pour la gestion des médecins, des patients et de la modération
- Une réduction mesurable du temps de recherche : de 20–30 minutes à moins de 2 minutes

En perspectives, plusieurs extensions pourraient enrichir la solution. L'intégration d'un système de notifications push via Firebase Cloud Messaging (FCM) améliorerait l'expérience de suivi des rendez-vous. L'ajout d'un système d'évaluation et d'avis permettrait aux patients de partager leur expérience. L'extension vers d'autres régions tunisiennes et la prise en charge d'une interface bilingue arabe/français renforcerait l'accessibilité de l'application. Enfin, l'intégration d'une intelligence artificielle pour la recommandation de médecins selon les antécédents médicaux ouvrirait la voie vers une santé numérique personnalisée.

Ce projet constitue ainsi une réponse concrète et innovante à la problématique de l'accessibilité aux soins de santé via le mobile, plaçant la technologie au service d'un parcours patient plus rapide, plus fiable et mieux adapté aux réalités contemporaines.